



北京邮电大学
Beijing University of Posts and Telecommunications

BUPT Compilers Lab5 2024Fall

Name: 项 枫

Student ID: 2022211570

Class Number: 2022211801

Container Number:

6eca9c2b53e9aaa0c65b6af6613814be05cc0d8ccc63b7df064c4a7230aab09f

1 Read Code

1.1 symtab.h

struct entry is used to describe entries in the symbol table. It has two fields. *key* stores symbol's name, and *value* stores symbol's value.

```
typedef struct entry {  
    char key[KEY_LEN+1];  
    VAL_T value;  
} entry;
```

1.2 symtab.ll.c

struct symtab implements the symbol table with a linked list.

```
struct symtab {  
    entry entry;  
    struct symtab *next;  
};
```

symtab_init, *symtab_insert*, *symtab_lookup*, and *symtab_remove* implement these operations on symbol tables with corresponding implementations on linked lists.

```
symtab *symtab_init(){  
    symtab *self = malloc(sizeof(symtab));  
    memset(self, '\0', sizeof(symtab));  
    self->next = NULL;  
    return self;  
}
```

```
int symtab_insert(symtab *self, char *key, VAL_T value){  
    symtab *ptr = self;  
    while(ptr->next != NULL){  
        if(strcmp(ptr->entry.key, key) == 0)  
            return 0;  
        ptr = ptr->next;  
    }  
    symtab *node = malloc(sizeof(symtab));  
    memset(node, '\0', sizeof(symtab));  
    entry_init(&node->entry, key, value);  
    node->next = NULL;  
    ptr->next = node;  
    return 1;  
}
```

```
VAL_T symtab_lookup(symtab *self, char *key){  
    symtab *ptr = self;
```

```
while(ptr != NULL){  
    if(strcmp(ptr->entry.key, key) == 0)  
        return ptr->entry.value;  
    ptr = ptr->next;  
}  
return -1;  
}
```

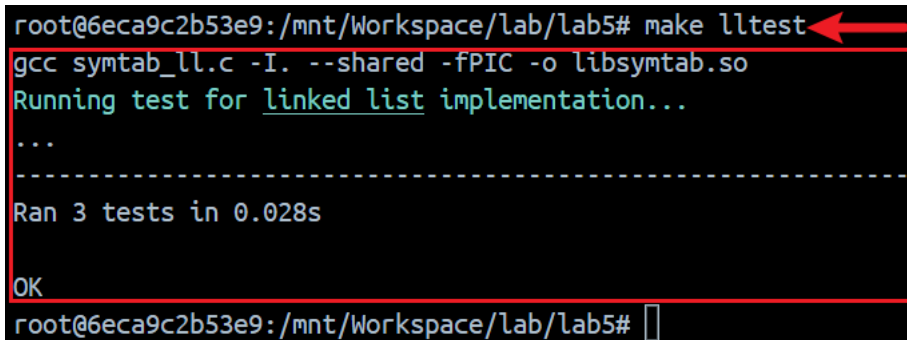
```
int symtab_remove(symtab *self, char *key){  
    symtab *ptr = self, *tmp;  
    while(ptr->next != NULL) {  
        if(strcmp(ptr->next->entry.key, key) == 0){  
            tmp = ptr->next;  
            ptr->next = ptr->next->next;  
            free(tmp);  
            return 1;  
        }  
        ptr = ptr->next;  
    }  
    return 0;  
}
```

2 Run

The command was used:

```
make lltest
```

The screenshot of the compilation and execution results.



A terminal window with a black background and white text. The prompt is 'root@6eca9c2b53e9:/mnt/Workspace/lab/lab5#'. The command 'make lltest' is entered, followed by the compilation command 'gcc symtab_ll.c -I. --shared -fPIC -o libsymtab.so'. The output shows 'Running test for linked list implementation...', three dots, a dashed line, 'Ran 3 tests in 0.028s', and 'OK'. A red arrow points to the 'make lltest' command, and a red box highlights the entire output section from 'Running test' to 'OK'. The prompt 'root@6eca9c2b53e9:/mnt/Workspace/lab/lab5#' is shown at the bottom.

```
root@6eca9c2b53e9:/mnt/Workspace/lab/lab5# make lltest  
gcc symtab_ll.c -I. --shared -fPIC -o libsymtab.so  
Running test for linked list implementation...  
...  
-----  
Ran 3 tests in 0.028s  
OK  
root@6eca9c2b53e9:/mnt/Workspace/lab/lab5#
```